COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	7/22/2026	**DATE SUBMITTED:**	7/22/2026
NAME:	Fernandez, Cedric S.	**INSTRUCTOR:**	Engr.JIMLORD QUEJADO

Seatwork 3.1: Creating a Singly Linked List

1. PROCEDURE OUTPUT

SINGLY FILE

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

//node creation
template <typename T>
class SingleList{
    public:
    T data; //this contains the data to be stored in the node
    SingleList<T>* next = nullptr;
};


template <typename T>
void ListTraversal(SingleList<T>* head) {
    if(head == nullptr) {
        std::cout<<"The list is empty\n";
        return;
    }

    while(head != nullptr){
        std::cout<<head->data;

        if(head->next != nullptr) {
            std::cout<<"->";
        }

        //GOES TO NEXT NODE n := next
        head = head->next;
    }

    std::cout<<std::endl;
}

template <typename T>
void sllinsertHead(T newData, SingleList<T>** currentHead) {

    //1. Allocate memory for the new node
    SingleList<T>* newNode = new SingleList<T>;

    //2. Put our data into the new node
    newNode->data = newData;

    //3. Set Next of the new node to point to the previous head
    newNode->next = *currentHead;

    //4. Reset Head to point to the new node
    *currentHead = newNode;

```

```cpp
}

template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode) {

    //1. Check if it is the head node (previous mode is null)
    if(prevNode == nullptr){
        //2. If null, print "Previous value cannot be null.""
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }
    //Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;

    newNode->next = prevNode->next;

    prevNode->next = newNode;

}

template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head) {

    //1. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;   // was: SingleList<T> newNode = new SingleList<T>;

    //2. Dereference to the head node
    SingleList<T>* currentNode = *head;

    //3. Store data in new node
    newNode->data = newData;
    //4. Point to Null
    newNode->next = nullptr;

    //5. Trverse the list until it is pointed to null
    while (currentNode->next != nullptr)
    {
        currentNode = currentNode->next;
    }

    //6. Point the next of the current node to the new node
    currentNode->next = newNode;

}
```

```cpp

template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head) {

    //if there is nothing to delete
    if (*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    //searching the node; 1. the node doesn't exist + we found the data
    while (currNode != nullptr && currNode->data != findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }
    //if data not found Currnode == nullptr in the while loop:
    if (currNode == nullptr) {
        std::cout<<"the data is not found \n" <<std::endl;
        return;
    }

    //if data was found:
    if(prevNode == nullptr){
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }
    delete currNode;
}

template <typename T>
void sllDeleteList(SingleList<T> **head) {
    SingleList<T>* current = *head;

    while(current != nullptr) {

        SingleList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
}

#endif
```

MAIN DRIVE

```cpp
#include<iostream>
#include "singly_ll.h"

int main(){

    //stack
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    //allocate memory to heap
    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;

    //Storing a value inside the node
    head -> data='C';
    second -> data='E';
    third -> data= 'D';
    fourth -> data = 'Y';

    //linking the nodes
    head-> next = second;
    second-> next= third;
    third-> next = fourth;
    fourth-> next = nullptr;

    std::cout<<"testing of Travelsal: " << std::endl;
    ListTraversal(head);

    std::cout<<"Testing of sllinsertHead: " << std::endl;
    sllinsertHead('C', &head);
    ListTraversal(head);

    std::cout<<"Testing of sllGeneralInsert: " << std::endl;
    sllGeneralInsert('E',head->next->next);
    ListTraversal(head);

    std::cout<<"Testing of sllinsertEnd: " << std::endl;
    sllinsertEnd('D', &head);
    ListTraversal(head);

    std::cout<<"Testing of deleting the node: " << std::endl;
    sllDeleteNode('Y', &head);
    ListTraversal(head);

    std::cout<<"deleting All nodes: " << std::endl;
    sllDeleteList(&head);
    ListTraversal(head);

    return 0;
}
```

COMPUTER ENGINEERING

QUEZON CITY

OUTPUT

```
PS C:\Users\csf18\Desktop\Codes\hoa3p1> cd "c:\Users\csf18\Desktop\Codes\hoa3p1\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
testing of Travelsal:
C->E->D->Y
Testing of sllinsertHead:
C->C->E->D->Y
Testing of sllGeneralInsert:
C->C->E->E->D->Y
Testing of sllinsertEnd:
C->C->E->E->D->Y->D
Testing of deleting the node:
C->C->E->E->D->D
deleting All nodes:
The list is empty
```

CONCLUSION & LEARNING

The things I learned from this assignment centered on creating and manipulating singly linked lists specifically in the part of node insertion from the head to the tail, as well as node deletion. Helped demonstrated how to insert, transfer, and remove data across created nodes, alongside other said techniques for instantiating and repositioning the nodes themselves. Additionally, it highlighted the vital role pointers play in dynamic list management, as well as the practical use of C++ templates and traversal algorithms to ensure proper memory allocation and data accessibility.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to polish my explanation to make it more clear and coherent**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.